

得物前端面经

得物前端面试一面

1. 实现防抖函数时，为什么建议用 `apply` 而不是直接调用函数？
2. React Hooks中 `useMemo` 和 `useCallback` 的本质区别是什么？
3. 手写：Promise.allSettled的polyfill
4. 浏览器事件循环中，`requestAnimationFrame` 的执行时机
5. 如何用CSS实现宽度自适应且保持宽高比1:1的容器？

得物前端面试二面

1. 实现虚拟列表时，如何计算可视区域外的缓冲区？
2. Webpack的tree-shaking原理及ES Module限制
3. 从输入URL到页面展示的全链路性能优化方案
4. 手写：带并发限制的异步调度器（最多同时运行2个任务）
5. 如何用Intersection Observer实现图片懒加载？
6. Vue3响应式原理中，为什么用Proxy替代defineProperty？
7. 设计一个前端灰度发布系统
8. 如何从0到1搭建前端监控体系？
9. 微前端方案落地时的样式隔离方案
10. 手写：大文件分片上传+断点续传
11. 解释浏览器渲染层合并（composite）优化原理

一面参考答案

1. 实现防抖函数时，为什么建议用apply而不是直接调用函数？

在防抖函数中，使用 `apply` 的主要原因是为了保持函数的上下文（`this`）和参数。通过 `apply`，可以将当前上下文和参数传递给目标函数，这样可以确保在调用时能够正确地引用到原始函数的上下文。

例如，在防抖函数中，如果直接调用函数，`this` 的上下文可能会丢失，导致函数无法正确执行。使用 `apply` 可以避免这个问题：

代码块

```
1  function debounce(func, wait) {
2    let timeout;
3    return function(...args) {
4      const context = this; // 保存上下文
5      clearTimeout(timeout);
6      timeout = setTimeout(() => {
7        func.apply(context, args); // 使用apply保持上下文和参数
8      }, wait);
9    };
10 }
```

2. React Hooks中useMemo和useCallback的本质区别是什么？

`useMemo` 和 `useCallback` 都是 React 的性能优化工具，但它们的用途和返回值不同：

- **useMemo**：用于记忆计算的值。它接受一个计算函数和依赖数组，如果依赖数组中的值没有变化，则返回上次计算的结果。它适用于需要进行昂贵计算的场景。

代码块

```
1  const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

- **useCallback**：用于记忆函数。它接受一个回调函数和依赖数组，如果依赖数组中的值没有变化，则返回上次创建的函数引用。它适用于避免子组件不必要的重新渲染。

代码块

```
1  const memoizedCallback = useCallback(() => {
2    doSomething(a, b);
3  }, [a, b]);
```

本质区别在于，`useMemo` 返回的是一个值，而 `useCallback` 返回的是一个函数。

3. 手写：Promise.allSettled的polyfill

`Promise.allSettled` 方法返回一个 Promise，只有当所有给定的 Promise 都已解决（无论是成功还是失败）时才会解决，返回一个对象数组，描述每个 Promise 的最终状态。

下面是 `Promise.allSettled` 的简单实现：

代码块

```
1  if (!Promise.allSettled) {
2    Promise.allSettled = function(promises) {
3      return new Promise((resolve) => {
4        const results = [];
5        let completedCount = 0;
6
7        promises.forEach((promise, index) => {
8          Promise.resolve(promise).then(
9            (value) => {
10              results[index] = { status: 'fulfilled', value: value };
11            },
12            (reason) => {
13              results[index] = { status: 'rejected', reason: reason
14            };
15          }
16        ).finally(() => {
17          completedCount++;
18          if (completedCount === promises.length) {
19            resolve(results);
20          }
21        });
22      });
23    };
24  }
```

4. 浏览器事件循环中，requestAnimationFrame的执行时机

`requestAnimationFrame` 是一种浏览器提供的 API，用于在浏览器下次重绘之前执行指定的回调函数。它的执行时机是在浏览器的渲染周期中，具体来说：

1. 浏览器会在每次重绘之前调用所有注册的 `requestAnimationFrame` 回调。
2. 这些回调会在浏览器绘制之前执行，允许开发者在下一帧中更新动画状态。
3. `requestAnimationFrame` 的调用会被压缩到每秒 60 帧（即每 16.67 毫秒）左右，具体取决于浏览器的刷新率。

因此，使用 `requestAnimationFrame` 可以提供更平滑的动画效果，因为它与浏览器的渲染周期紧密集成。

5. 如何用CSS实现宽度自适应且保持宽高比1:1的容器？

可以使用 CSS 的 `padding-top` 技巧来实现一个宽度自适应且保持 1:1 高宽比的容器。具体实现如下：

代码块

```
1 <div class="square"></div>
```

代码块

```
1 .square {  
2     width: 100%; /* 宽度自适应 */  
3     padding-top: 100%; /* 高度等于宽度的 100% */  
4     background-color: lightblue; /* 背景色可选 */  
5     position: relative; /* 相对定位以便放置子元素 */  
6 }
```

在这个例子中，`padding-top: 100%` 会使得容器的高度与宽度相等，从而保持 1:1 的高宽比。这个技巧利用了 CSS 中 `padding` 的百分比是相对于元素的宽度计算的特性。

二面参考答案：

1. 实现防抖函数时，为什么建议用apply而不是直接调用函数？

在防抖函数中，使用 `apply` 的主要原因是为了保持函数的上下文（`this`）和参数。通过 `apply`，可以将当前上下文和参数传递给目标函数，这样可以确保在调用时能够正确地引用到原始函数的上下文。

例如，在防抖函数中，如果直接调用函数，`this` 的上下文可能会丢失，导致函数无法正确执行。使用 `apply` 可以避免这个问题：

代码块

```
1  function debounce(func, wait) {
2    let timeout;
3    return function(...args) {
4      const context = this; // 保存上下文
5      clearTimeout(timeout);
6      timeout = setTimeout(() => {
7        func.apply(context, args); // 使用apply保持上下文和参数
8      }, wait);
9    };
10 }
```

2. React Hooks中useMemo和useCallback的本质区别是什么？

`useMemo` 和 `useCallback` 都是 React 的性能优化工具，但它们的用途和返回值不同：

- **useMemo**：用于记忆计算的值。它接受一个计算函数和依赖数组，如果依赖数组中的值没有变化，则返回上次计算的结果。它适用于需要进行昂贵计算的场景。

代码块

```
1  const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
```

- **useCallback**：用于记忆函数。它接受一个回调函数和依赖数组，如果依赖数组中的值没有变化，则返回上次创建的函数引用。它适用于避免子组件不必要的重新渲染。

代码块

```
1  const memoizedCallback = useCallback(() => {
2    doSomething(a, b);
```

```
3   }, [a, b]);
```

本质区别在于，`useMemo` 返回的是一个值，而 `useCallback` 返回的是一个函数。

3. 手写：Promise.allSettled的polyfill

`Promise.allSettled` 方法返回一个 Promise，只有当所有给定的 Promise 都已解决（无论是成功还是失败）时才会解决，返回一个对象数组，描述每个 Promise 的最终状态。

下面是 `Promise.allSettled` 的简单实现：

代码块

```
1  if (!Promise.allSettled) {
2    Promise.allSettled = function(promises) {
3      return new Promise((resolve) => {
4        const results = [];
5        let completedCount = 0;
6
7        promises.forEach((promise, index) => {
8          Promise.resolve(promise).then(
9            (value) => {
10              results[index] = { status: 'fulfilled', value: value };
11            },
12            (reason) => {
13              results[index] = { status: 'rejected', reason: reason
14            };
15          }
16        ).finally(() => {
17          completedCount++;
18          if (completedCount === promises.length) {
19            resolve(results);
20          }
21        });
22      });
23    };
24  }
```

4. 浏览器事件循环中，requestAnimationFrame的执行时机

`requestAnimationFrame` 是一种浏览器提供的 API，用于在浏览器下次重绘之前执行指定的回调函数。它的执行时机是在浏览器的渲染周期中，具体来说：

1. 浏览器会在每次重绘之前调用所有注册的 `requestAnimationFrame` 回调。
2. 这些回调会在浏览器绘制之前执行，允许开发者在下一帧中更新动画状态。
3. `requestAnimationFrame` 的调用会被压缩到每秒 60 帧（即每 16.67 毫秒）左右，具体取决于浏览器的刷新率。

因此，使用 `requestAnimationFrame` 可以提供更平滑的动画效果，因为它与浏览器的渲染周期紧密集成。

5. 如何用CSS实现宽度自适应且保持宽高比1:1的容器？

可以使用 CSS 的 `padding-top` 技巧来实现一个宽度自适应且保持 1:1 高宽比的容器。具体实现如下：

代码块

```
1 <div class="square"></div>
```

代码块

```
1 .square {  
2     width: 100%; /* 宽度自适应 */  
3     padding-top: 100%; /* 高度等于宽度的 100% */  
4     background-color: lightblue; /* 背景色可选 */  
5     position: relative; /* 相对定位以便放置子元素 */  
6 }
```

在这个例子中，`padding-top: 100%` 会使得容器的高度与宽度相等，从而保持 1:1 的高宽比。这个技巧利用了 CSS 中 `padding` 的百分比是相对于元素的宽度计算的特性。

6. Vue3响应式原理中，为什么用Proxy替代defineProperty?

在 Vue 3 中，使用 `Proxy` 替代 `defineProperty` 的原因主要有以下几点：

1. **支持深层次的响应式：** `Proxy` 可以直接代理整个对象，而 `defineProperty` 只能对对象的属性进行代理。使用 `Proxy` 可以轻松地实现对嵌套对象的深层响应式。
2. **性能优化：** 使用 `Proxy` 可以避免在每次属性访问时都需要使用 `defineProperty` 进行 getter 和 setter 的定义，从而提高性能。
3. **简化代码：** `Proxy` 提供了更简单的 API，可以拦截各种操作（如读取、写入、删除等），使得实现响应式的代码更加简洁。
4. **动态拦截：** `Proxy` 可以动态拦截对象的操作，而 `defineProperty` 需要在对象创建时就定义好属性，灵活性不足。